

UNIVERSITY OF TECHNOLOGY EINDHOVEN

PROJECT 2

5LSH0 COMPUTER VISION AI AND 3D DATA ANALYSIS

Anomaly Detection With Autoencoder

Names

Joost LELIVELD

Student numbers

1609726

June 18, 2026

Contents

1	Basic structure	2
2	Questions	2
2.1	Adding noise	2
2.2	Training	2
2.3	Corruption in training	3
2.4	SSIM Implementation	3
2.5	Deeper Autoencoder	4
2.6	Convolutional Autoencoder	5
2.7	Best implementation and overfitting	6
2.8	Tresholding	7
2.9	Confusion matrix	7
2.10	Precision, Recall and F1-score	8
2.11	ROC curve and AUC	8
2.12	False alarm rate	8
2.13	Cons of autoencoder structure	9
2.14	Alternatives	9

1 Basic structure

The basic structure can be found in the notebook. Below that also functions are defined that are needed in several questions. To prevent duplicate code I have combined those, but I think it is still clear how every question was answered with those functions.

2 Questions

2.1 Adding noise

The code for adding the noise can be found in the notebook. The pixel values were multiplied with random values and then clamped to 0 and 1 again. With the noise factor the level of change of the pixels can be changed. At first I put the noise level at 0.3, but then the model was able to detect the anomalies too well. So now it is 0.15.

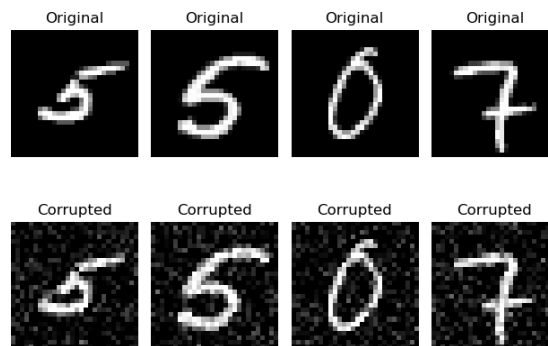


Figure 1: 4 samples from test data set with noise level 0.15

2.2 Training

I expected the clean images to be reconstructed well, with some blur added since it is a simple model with dimensionality reduction. Since autoencoders learn a compressed representation of the image, potentially leading to loss of details. The model was not trained at all on corrupted images so I did expect there to be a worse reconstruction, with more blur and more general features. The model is trained to minimize the MSE between the input and the reconstructed image, so it will try to recreate the structure of an expected pattern. It will expect the structure of an image without noise. The corrupted and non corrupted images have more similar output than I personally expected, but the clean samples do have a better reconstruction where the original number is visible more clearly. And the noise added to the picture has disappeared in the reconstruction, which would be visible in the MSE. The difference in the results would probably grow for a model that is fitted even more to the clean training data, this was a simple implementation and needed a lot of training for this result. Firstly I used a standard training procedure which had the learning rate at 0.1 for 10 epochs which produced a blurry reconstruction that looks similar to an 8 or a 3. This could be because the model learned a basic structure that would work well on the loss function, without learning the details and intrinsic structure of the data. Putting the learning rate really high sped up the training process by a lot which allowed for better results. Still putting it this high is probably not ideal as it will cause oscillations when finding minima.

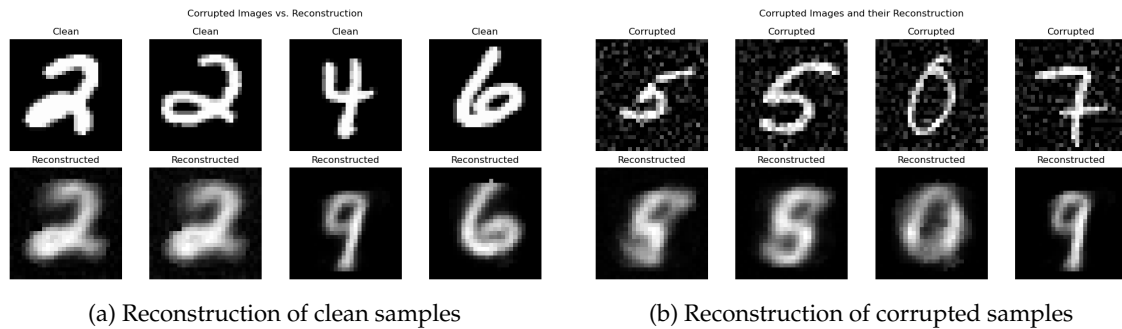


Figure 2: Comparison of reconstructions for basic auto encoder and MSE loss function

2.3 Corruption in training

The model will then also have to account for corruption to get a clean reconstruction. Depending on the level of corruption as well as the amount of images that are corrupted it could make the model perform better on general datasets, but worse on the specific clean datasets. Since the encoder learns a latent space that represents both clean and corrupted data. It will then consist of the essential features for clean and corrupted images. This also causes the difference in reconstruction error between clean and corrupted data to decrease. Since it will get better at reconstructing noisy image and probably worse at reconstructing clean images. This smaller difference in MSE will make it challenging to use a threshold for anomaly detection.

2.4 SSIM Implementation

With MSE loss the focus is on individual pixel differences between images, this could result in blurry images that do not capture the structural aspects of the image. It is sensitive to small variations. SSIM is more focused on structures, edges, and contrast. This is more aligned with our visual perception. So SSIM compares image quality more in general rather than pixel wise. This is in line with the results, since the overall quality of the reconstructed images seems higher when using the SSIM loss function. However in both cases more training could lead to different results.

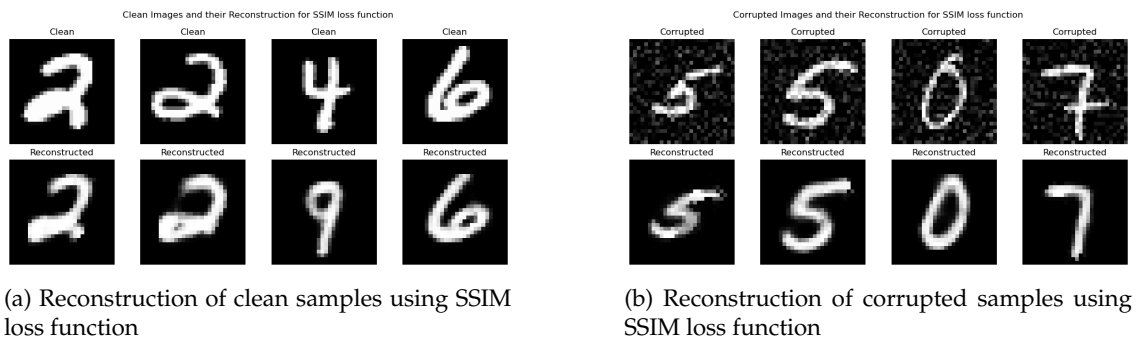


Figure 3: Comparison of reconstructions with SSIM as loss functions

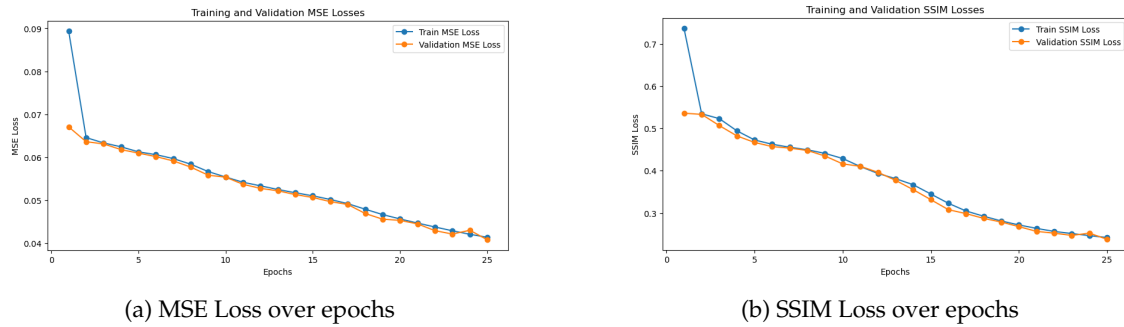


Figure 4: Comparison of MSE and SSIM losses during training.

2.5 Deeper Autoencoder

The improvements to the basic auto encoder were:

1. **Deeper Architecture:** I added additional layers, to learn more complex representations of the training data. The size of the latent space remained the same for a fair comparison.
2. **Batch Normalization:** Batch normalization helps to stabilize the training by normalizing the data after each layer. For every layer the data is transformed to be of a norm scale and of uniform distribution. It is done after the activation functions, so no extra parameters have to be trained and learned.
3. **Adam Optimizer:** Instead of the stochastic gradient descent I decided to use the Adam optimizer. On the previous auto encoder a big issue I struggled with was the learning rate, which for the Adam optimizer is adaptive. Which makes this model have a faster and better convergence.

It is clearly visible that the convergence when training happens a lot faster than the training for the basic autoencoder. The loss values are already lower after 2 epochs for the deeper auto encoder, compared to the 25 epochs of the basic auto encoder. The biggest difference for this is probably the Adam optimizer, instead of SGD. The results are also clearly visible in the reconstruction. The reconstructed images for the deeper autoencoder are a lot less blurry and contain more detail. It is very clear most of the information is retained, while for the basic autoencoder this was not the case.

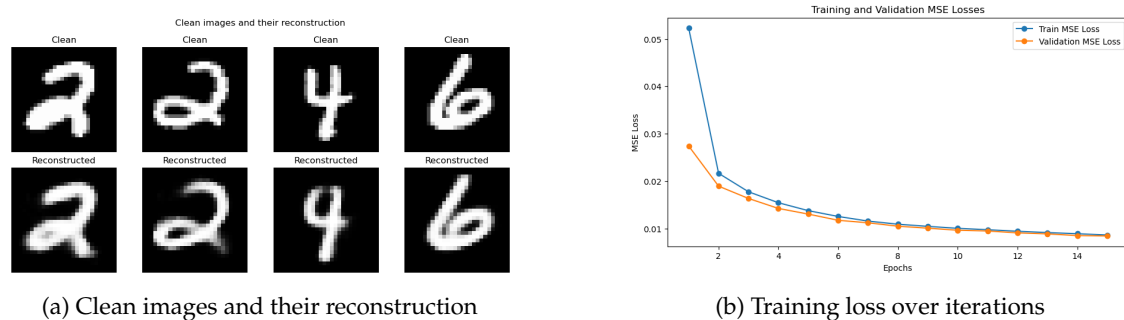


Figure 5: Comparison of clean images with reconstruction and training loss graph

To check if the right noise levels were implemented the reconstruction of noisy images are also tested. This also shows a better and more detailed reconstruction then for the basic auto encoder. It does contain more detail, but the reconstruction is definitely not correct, which would make it possible to use as for anomaly detection.

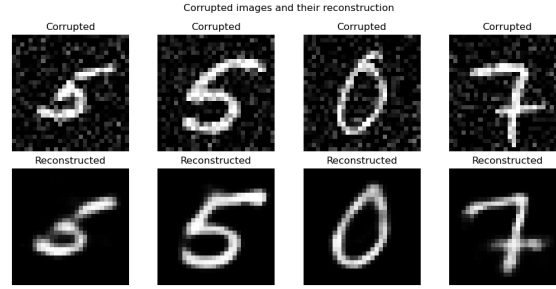


Figure 6: Reconstruction of corrupted images for deeper auto encoder

2.6 Convolutional Autoencoder

The convolutional encoder I implemented uses convolutional layers (CL) with feature maps (from 1 to 32 channels) and similar to the 2 previous encoders it also uses ReLU activations, batch normalization, and fully connected layers. BN is applied to the deeper layers where it usually has the most impact on convergence. Each image is reduced to a latent representation of size 4.

The structure of the layers is as follows

Encoder Layers

$CL_i(\text{input channels, output channels, kernel size, stride, padding})$

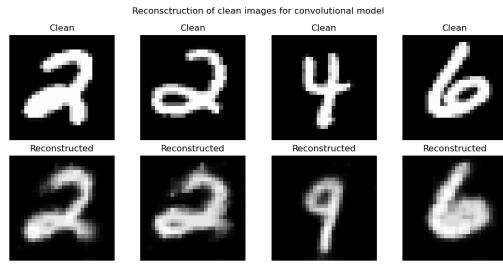
$FC_i(\text{input size, output size})$

- CL_1 : (1 (grayscale image), 8, 3×3 , 2, 1). Input: $(1, 28 \times 28) \rightarrow$ Output: $(8, 14 \times 14)$.
- ReLU activation
- CL_2 : (8, 16, 3×3 , 2, 1). Input: $(8, 14 \times 14) \rightarrow$ Output: $(16, 7 \times 7)$.
- ReLU activation
- CL_3 : (16, 32, 3×3 , 2, 0). Input: $(16, 7 \times 7) \rightarrow$ Output: $(32, 3 \times 3)$.
- Batch normalization
- ReLU activation
- Flatten. Input: $(32, 3 \times 3) \rightarrow$ Output: $(32 \times 3 \times 3 = 288)$.
- FC_1 : (288, 128). Input: 288 $\rightarrow$ Output: 128.
- FC_2 : (128, 4). Input: 128 $\rightarrow$ Output: 4 (latent space).

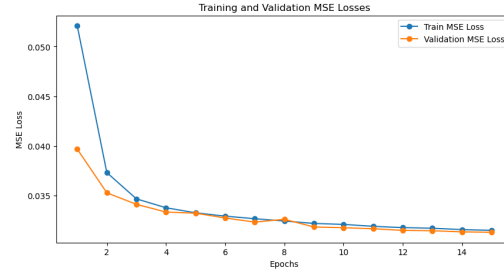
Decoder Layers

- FC_3 : (4, 128). Input: 4 (latent space) $\rightarrow$ Output: 128.
- FC_4 : (128, 288). Input: 128 $\rightarrow$ Output: 288.
- Unflatten. Input: (288) $\rightarrow$ Output: $(32, 3 \times 3)$.
- TCL_1 : (32, 16, 3×3 , 2, 0, 0). Input: $(32, 3 \times 3) \rightarrow$ Output: $(16, 7 \times 7)$.
- Batch normalization

- ReLU activation
- TCL_2 : (16, 8, 3×3 , 2, 1, 1). Input: (16, 7×7) $\rightarrow$ Output: (8, 14×14).
- Batch normalization
- ReLU activation
- TCL_3 : (8, 1 (grayscale image), 3×3 , 2, 1, 1). Input: (8, 14×14) $\rightarrow$ Output: (1, 28×28).
- Sigmoid activation



(a) Reconstruction of convolutional images



(b) Training loss for convolutional network

Figure 7: Comparison of convolutional reconstruction and training loss.

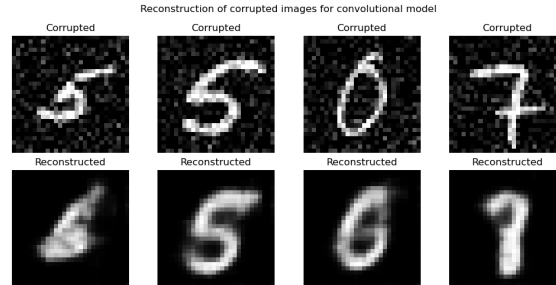


Figure 8: Reconstruction of corrupted samples for convolutional auto encoder

2.7 Best implementation and overfitting

The best results were found for the deeper autoencoder model. The lossfunctions are visible in the figure of that section. Overfitting happens when a model learns the training data too well, sometimes to it could capture noise or random fluctuations in the training data, and lose the underlying pattern. The consequence of this is that the model will perform poorly on unseen data (bad generalization on validation set or test set). Overfitting can be detected when the training loss continues to decrease over the epochs, but the validation loss starts to increase or plateau. This causes a larger and growing difference between training and validation losses, which suggests overfitting. One solution to this is 'early stopping'. This means to stop the training process before the variance of loss between the sets start to increase. Stopping too early could however also cause underfitting. Other ways to prevent overfitting are:

- Pruning: Only selecting relevant features will reduce the model learning noise and will also reduce model complexity.
- Regularization: Penalizing large weight values will reduce model complexity and reliance on specific neurons.

- Data augmentation: Expanding the dataset by applying transformations or rotations will make the model robust for more cases.
- Dropout: By randomly disabling neurons during training the model learns to adapt and becomes more general.

For the next questions I used the deeper auto encoder model which was stopped after 5 epochs.

2.8 Thresholding

As can be seen in the picture the threshold I have chosen is error = 0.0188. This is where the lines of the error distributions intersect. Right now quite a significant part of the clean images still has an error rate above that threshold. However increasing the threshold (moving the threshold further to the right) would cause too many corrupted samples to be classified as clean samples. This is because the distribution is increasing quite steeply on that part, meaning a lot of the samples fall near that region. So this threshold takes the middle road to have not too many corrupted samples below the threshold, while accepting that some clean samples will fall below the threshold.

(Before this the noise level was set at 0.3 and then the distributions did not overlap. I know this threshold will not give a really good model, but it will help answer the next questions. Also the choice of threshold could be improved based on the F1-score or ROC curve, but that is asked in later questions. So I decided to answer more intuitively.)

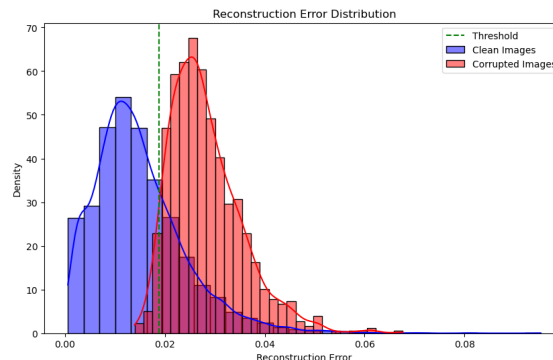


Figure 9: Error distribution for clean and corrupted images on the deeper auto encoder model

2.9 Confusion matrix

- True Positives (TP): 965
- True Negatives (TN): 6570
- False Positives (FP): 2430
- False Negatives (FN): 35

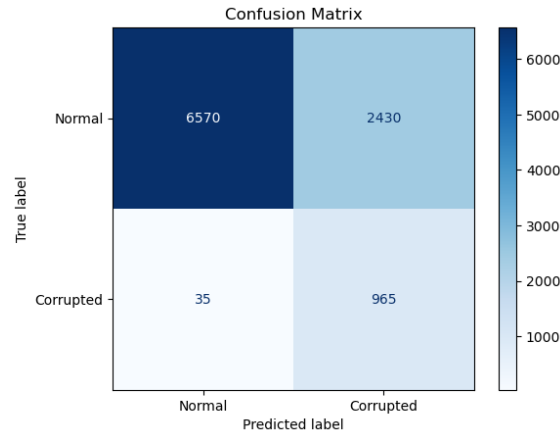


Figure 10: Confusion matrix for threshold 0.0188

2.10 Precision, Recall and F1-score

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{965}{965 + 2430} \approx 0.28$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{965}{965 + 35} \approx 0.97$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \cdot \frac{0.28 \cdot 0.97}{0.28 + 0.97} \approx 0.44$$

The F1 score is the harmonic mean of the precision and recall, since the Recall is already high the best way to improve the F1 score would be to improve the precision. By looking at the formula for precision mentioned above you can see that it can be improved by reducing the false positive rate. In this case that means to reduce the amount of pictures that are labeled as corrupted while it are clean images. Increasing the threshold would solve this. Less images would be classified as corrupted, however this would decrease the recall. Right now the recall is very good, which means there are not a lot of cases where the model labels the image as clean while it is corrupted. With a higher threshold more corrupted images will have an error score below the threshold and thus be labeled as clean. For a higher F1-score it is thus important to find a balance between the recall and precision by changing the threshold.

2.11 ROC curve and AUC

The Receiver operating characteristic curve shows the performance of the model for different threshold values. The recall is plotted on the y-axis and the false positive rate (FPR/ FAR in next question) on the x-axis, for all thresholds where the range of both axis goes to 1. This shows the trade-off for the thresholds. Ideally the model goes through the top left corner (recall = 1, FPR = 0). The AUC is independent of the thresholds and calculates a single metric for the models ability at classifying across all thresholds. Where 1 equals perfect classification and 0.5 equals random guessing. It is derived from the ROC curve and can be used to compare models (higher AUC means better classification). The ROC curve can be used to analyze the trade off for a low FPR or high recall. Which one is more important depends on the use case, and the ROC can thus help with choosing the threshold in the region that is more important for that specific use case.

2.12 False alarm rate

$$\text{FAR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = \frac{2430}{2430 + 6570} \approx 0.27$$

The False Alarm Rate (FAR) is the proportion of true negatives that are classified as false positives. In this case it would be clean images that were classified as corrupted images. The ideal false alarm rate would of course be 0, you don't want to misclassify cases. However for the best model now it is 0.27. In anomaly detection for systems where detecting a positive causes great disruptions it would be better to have a really low alarm rate and maybe have a few more false negatives. But for cases like medical diagnosis then people are willing to have a higher false alarm rate since false negatives have greater consequences.

2.13 Cons of autoencoder structure

Autoencoders can be very dependent on the distribution of the data. The compression and reconstruction is based on the patterns in the data, so it needs enough to recognize these while still being generalizable. In anomaly detection the dataset needs to be complete and work well on all normal cases. If one case is underrepresented in the dataset and will always give a high reconstruction error then it could be classified as an anomaly. While having a too diverse dataset with general patterns that are hard to recognize will cause difficulty with selecting the threshold. A poor performance on normal cases will lead to a lot of overlap in the error distribution on normal cases and anomalies thus leading to difficulty in setting the threshold and low accuracy. And since setting this threshold is very important for the performance of the model also enough anomalies must be present for determining this threshold. This could be difficult since anomalies can be rare. Lastly the autoencoder structure lacks interpretability. The hidden layers and discovered patterns can not be explained. So it is unable to tell why something was marked as an anomaly which reduces the transparency needed in some systems.

2.14 Alternatives

The first alternative would be to train a convolutional neural network to classify clean and corrupted images. By preparing a labeled dataset with normal and corrupted images, then training that model to do binary classification on those images using the labels. With the same metrics as before the model could be evaluated. This is a more direct approach than encoding the data and does direct predictions without the need for thresholding. However it does require to have labeled training data with enough labels for anomalies. If this was not the answer you were looking for another method would be to use the k-nearest-neighbour algorithm and classify a datapoint based on the distance to the normal datapoints. However this also requires setting a threshold as to what distance would be considered an outlier.

The second alternative is to use a common Isolation Forest (IF). This is an unsupervised anomaly detection method, which uses random forest algorithms or decision trees to detect anomalies. It requires to image flattening and then training on features of normal images. Several decision trees are constructed from subsets of the data to have diverse trees. A tree is used to split the data, for each node a feature and a value is selected to split the data. The data are imaged are partitioned until the specific data point is isolated, the number of splits needed is an indication for the probability of a point being an anomaly. In general anomalies will have shorter path lengths as they should fall into smaller partitions quicker. By averaging the path length and thresholding that value one can classify a point as an anomaly or not. This works unsupervised, is efficient for high dimensional data and does not require a balanced dataset. Limitations are the need for a threshold, subtle anomaly detection and possible loss of spatial information.